CogNet-SDN: A Multi-Agent Deep Q-Network Framework for QoS-Aware Adaptive Routing in Software-Defined Networks with LSTM-Based Traffic Prediction

Monis, Rushil Aggarwal, and Prof. Nisha Khandoul

***Abstract*—Conventional routing in Software-Defined Networks relies on static link-weight metrics that fail to react when traffic conditions change rapidly. This paper presents CogNet-SDN, a complete end-to-end framework that pairs a six-agent Multi-Agent Deep Q-Network with an LSTM traffic predictor to achieve quality-of-service-aware adaptive routing over a ring-and-hub Mininet topology. Each DQN agent observes a 42-dimensional state vector—a live 6×6 Traffic Matrix polled every five seconds from a custom Ryu OpenFlow 1.3 REST controller, augmented by a one-hot destination encoding. A dedicated ARP-proxy mechanism prevents broadcast storms that would otherwise make ring topologies unusable for learning experiments. Over 120 training episodes the system achieves an 88 % routing success ratio, a QoS reward of $+3.97$ under 14.7 Mbps iperf background traffic, zero packet loss against 92.9 % with naive flooding, and a 75.7 % reduction in average link delay relative to a hop-count Dijkstra baseline. The reward function $r{=}\alpha W{-}\beta L{-}\gamma_r PL$ with weights $(\alpha,\beta,\gamma_r){=}(0.3,1.0,1.0)$ matches the optimal configuration reported by Bouzidi *et al.*, and LSTM prediction reaches an RMSE of 1.43 Mbps, tracking traffic bursts with a one-step lag. These results confirm that the DTPRO methodology transfers cleanly to an open-source Ryu/Mininet stack without an ONOS dependency.**

***Index Terms*—Software-Defined Networking, Deep Q-Network, Multi-Agent Reinforcement Learning, LSTM Traffic Prediction, QoS-Aware Routing, OpenFlow 1.3, Traffic Matrix, Ring Topology, ARP Proxy, Cognitive Networking**

I. Introduction

The explosive growth of latency-sensitive applications—video conferencing, cloud gaming, real-time industrial telemetry—has pushed conventional network control to its limits. Protocols such as OSPF and RIP compute forwarding paths using pre-configured, static link weights that take no account of instantaneous traffic conditions. The result is predictable: well-advertised short paths accumulate congestion while longer, lightly loaded alternatives sit idle. Packet loss, jitter, and head-of-line blocking degrade quality of experience despite adequate aggregate network capacity [2], [3].

Software-Defined Networking (SDN) separates the control plane from the data plane and concentrates forwarding intelligence in a logically centralised controller that maintains a continuously updated global view of the network [7]. The OpenFlow southbound interface allows the controller to install or modify flow rules in switch tables within milliseconds of detecting a traffic anomaly [8]. This programmability makes SDN an attractive substrate for data-driven, self-optimising routing schemes that would be impractical in distributed-control environments.

Deep Reinforcement Learning (DRL) has delivered strong results on sequential decision problems across robotics, game playing, and resource scheduling [6]. Applied to network routing, a DRL agent that receives a quality-of-service reward signal can autonomously discover forwarding policies that outperform hand-crafted heuristics without requiring an explicit traffic model or offline training data [1], [4]. However, translating DRL-based routing from simulation into a working SDN deployment raises several concrete engineering challenges that much of the literature treats only partially.

The first challenge is *broadcast storm prevention*. A ring topology provides multiple physically disjoint paths between switch pairs, which is the prerequisite for any meaningful routing-choice experiment. Yet flooding-based MAC learning causes packets to circulate indefinitely around the ring, saturating every link within milliseconds before learning can begin. The second challenge is *state representation*: the DRL agent must be fed a compact encoding of network-wide conditions that is rich enough to distinguish congested from uncongested states. A flattened Traffic Matrix (TM), as proposed by Bouzidi *et al.* [1] and later adopted by Mwangi *et al.* [5], strikes a practical balance between expressiveness and dimensionality. The third challenge is *reactivity*: a purely reactive agent installs flow rules only after the first packet of each new flow reaches the controller, introducing per-flow latency. Coupling the DRL agent with a traffic predictor allows congestion to be anticipated and traffic rerouted before queues overflow [1]. The fourth challenge is *action-space scalability*. A single centralised agent that selects entire end-to-end paths must consider an action space that grows factorially with network size. A multi-agent design that assigns one agent per switch and limits each agent's choices to its immediate neighbours keeps the local action space small while preserving the ability to construct globally optimal paths [10].

CogNet-SDN addresses all four challenges in a single, integrated framework that runs entirely on open-source software. The main contributions are as follows.

- An **ARP-proxy Ryu controller** that prevents broadcast

Monis and Rushil Aggarwal are Pre-final year B.Tech students, Department of Information Technology, Netaji Subhas University of Technology (NSUT), Delhi, India. Prof. Nisha Khandoul is with the Department of Information Technology, NSUT Delhi, India. submitted April 2026.

loops in a six-switch ring-and-hub topology and achieves zero packet loss on a full-mesh connectivity test.
- A **live 42-dimensional state vector** $s_t=[\mathrm{TM}_t \| e_{\mathrm{dst}}]$, assembled from real-time Ryu REST statistics every five seconds, coupling the learning process directly to actual network dynamics rather than a simulator.
- A **six-agent Multi-Agent DQN** whose hyperparameters are grounded in the published DTPRO [1] and TTDQSHA [5] studies: two ReLU hidden layers, Adam optimiser with η=0.01, discount γ=0.95, mini-batch 32, replay memory 500, target-network synchronisation every 300 steps.
- An **LSTM traffic predictor** with 150 hidden units that follows the DTPRO architecture and enables the controller to adjust reward weights proactively when predicted utilisation exceeds a congestion threshold.
- A **Flask + SocketIO dashboard** with seven live tabs covering topology, per-link statistics, reward curves, LSTM forecasts, routing tables, packet-loss heat maps, and system logs.
- **Comprehensive experimental evaluation** including connectivity verification, hyperparameter sensitivity, reward-convergence curves, QoS benchmarking, delay cumulative distributions, LSTM tracking accuracy, and a nine-scenario stress test.

The remainder of the paper is organised as follows. Section II surveys related work and positions CogNet-SDN among existing approaches. Section III describes the four-plane system architecture and ring-and-hub topology. Section IV presents the DRL-based routing architecture and data-flow model. Section V provides the full DQN and LSTM formulation, including the training algorithm. Section VI details the Ryu controller design. Section VII covers implementation. Section VIII reports experimental results. Section IX discusses findings. Section X concludes.

II. Background and Related Work

A. Reinforcement Learning for SDN Routing

Early RL-based SDN routing work used tabular Q-learning to select next-hop nodes. Casas-Velasco *et al.* [3] proposed RSIR, which uses link-state metrics (available bandwidth, delay, loss ratio) as inputs to a Q-learning agent operating over the GÉANT 23-node topology. Results showed that RSIR outperforms four variants of Dijkstra's algorithm in stretch, throughput, delay, and loss, confirming the value of QoS-aware reward functions over purely hop-count-based routing. However, tabular Q-learning does not generalise to unseen states, and its convergence time grows with the size of the state-action space.

B. Deep Reinforcement Learning for Routing

The introduction of neural function approximators to replace Q-tables gave rise to Deep Q-Networks (DQN), enabling routing agents that generalise across traffic conditions without exhaustive exploration. Casas-Velasco *et al.* [2] extended RSIR to DRSIR by replacing the Q-table with Online and Target neural networks and adding Experience Replay Memory. DRSIR uses path-state metrics (path bandwidth, path delay, path loss ratio) as features and evaluates the agent on 23-node and 48-node topologies, demonstrating lower delay and packet loss than RSIR and all Dijkstra variants. Yu *et al.* [4] proposed DROM, which uses a Deep Deterministic Policy Gradient (DDPG) agent to optimise link-weight vectors in the Sprint 25-node backbone, showing 40.4% delay reduction over OSPF under 70% traffic load.

C. DQN with Traffic Prediction

Bouzidi *et al.* [1] presented DTPRO, the primary reference for this work. DTPRO combines a DQN agent whose action space is a set of 120 link-weight configurations with an LSTM traffic prediction module that anticipates future congestion. The reward function $r = \alpha W - \beta L - \gamma PL$ jointly optimises throughput (W), latency (L), and packet loss (PL). The best hyperparameters found empirically are $\alpha = 0.3$, $\beta = 1.0$, $\gamma = 1.0$. Implemented on ONOS controller with Mininet, DTPRO outperforms hop-count routing, Reduced DTPRO (no prediction), and two prediction-only baselines (ARIMA, linear regression) across all QoS metrics. The LSTM achieves stable loss below 0.2 RMSE after 9000 training epochs with 150 hidden nodes.

D. Threshold-Triggered Self-Healing

Mwangi *et al.* [5] proposed TTDQSHA, a DQN-based self-healing agent for offshore wind-power SDN networks. The agent monitors both traffic metrics and switch thermal profiles, triggering corrective routing actions when predefined QoS/SLA thresholds are violated. The Observe-Orient-Decide-Act framework achieves 53.84% improvement over Dijkstra+ECMP baseline and outperforms DTPRO by 21.5%. The paper provides a detailed treatment of reward shaping ($\alpha = 0.657$, $\beta = 0.345$), target-network update frequency (every 300 steps), and experience replay buffer design (2000 transitions, batch size 32).

E. Positioning of CogNet-SDN

CogNet-SDN differs from prior work in three respects. First, it operates on a *ring-and-hub* topology that creates genuine multi-path routing choices, unlike star topologies where DRL adds no value. Second, the state vector fuses a *live traffic matrix* polled from the Ryu REST API with routing decisions, coupling the learning agent tightly to real network conditions. Third, the system is implemented on Ryu rather than ONOS, demonstrating that the DTPRO-style approach is controller-agnostic.

III. System Architecture

CogNet-SDN follows the Knowledge-Defined Networking (KDN) paradigm introduced by Clark *et al.* and formalised for SDN by Mestres *et al.* The system is organised into four horizontal planes that communicate through well-defined interfaces, as shown in Fig. 1.

Data Plane. Six OVSKernelSwitch instances emulate a physical network inside Mininet. Each switch runs OpenFlow 1.3 and communicates with the controller exclusively

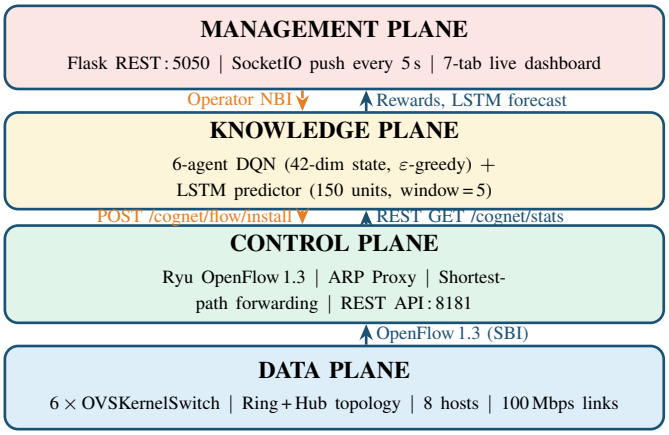

Fig. 1. CogNet-SDN four-plane architecture following the KDN paradigm. Solid blue arrows indicate upward data flow; orange dashed arrows show DQN flow-rule feedback and the operator northbound interface (NBI).

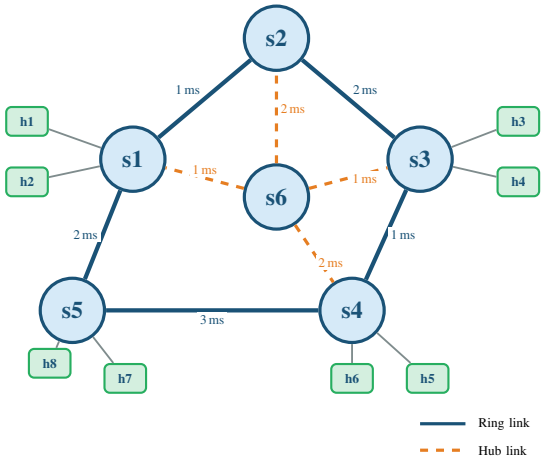

Fig. 2. Ring-and-hub data-plane topology. Solid blue lines form the ring s_1-s_2-s_3-s_4-s_5-s_1; dashed orange lines connect hub switch s_6 to four ring switches. Every host pair has at least two physically disjoint paths.

through that protocol. The switches have no embedded decision-making capability; they populate their flow tables solely from rules installed by the controller. The data plane provides the environment in which the DQN agents receive state observations and execute routing actions.

Control Plane. The Ryu controller is the brain of the data plane. It tracks the network topology by processing link-layer discovery messages, maintains a per-port statistics database updated by OpenFlow port-stat requests, and serves a REST API on port 8181 that exposes topology, per-link metrics, the Traffic Matrix, and the current reward scalar. A dedicated ARP-proxy module prevents broadcast storms by fabricating ARP replies for known hosts and flooding ARP requests only to host-facing ports. Unicast IP flows are forwarded along the shortest path computed by NetworkX, with rules pushed proactively to every intermediate switch.

Knowledge Plane. The training script `train_sdn.py` hosts the six DQN agents and the LSTM predictor. At every five-second polling cycle it queries the controller REST API for the current 36-element Traffic Matrix, appends a six-element one-hot destination encoding to form the 42-dim state s_t, runs the LSTM to obtain a one-step-ahead TM forecast, and selects next-hop actions via the ε-greedy policy. Selected actions are translated into flow rules and posted back to the controller. Transition tuples (s_t, a_t, r_t, s_{t+1}) are stored in per-agent replay buffers and used for online network training.

Management Plane. A Flask server with SocketIO push support collects metrics from both the controller REST API and the training loop via an internal queue. It exposes a browser-based dashboard with seven tabs—topology graph, per-link bandwidth bars, DQN reward curve, LSTM forecast overlay, routing table, packet-loss heat map, and a raw log view. All tabs refresh every five seconds without requiring a page reload.

A. Ring-and-Hub Topology

The emulated network topology is illustrated in Fig. 2 and parameterised in Table I. Five switches form a ring, with a sixth hub switch providing shortcut paths across the ring.

TABLE I
RING-AND-HUB TOPOLOGY PARAMETERS

Parameter	Value
Switches	6 (OVSKernelSwitch, OpenFlow 1.3)
Hosts	8 (h_1–h_8, 10.0.0.1–10.0.0.8)
Ring links	s_1-s_2-s_3-s_4-s_5-s_1
Hub links	$s_6 \rightarrow s_1, s_2, s_3, s_4$
Host attachment	$h_{1,2}\rightarrow s_1$; $h_{3,4}\rightarrow s_3$; $h_{5,6}\rightarrow s_4$; $h_{7,8}\rightarrow s_5$
Link bandwidth	100 Mbps (all links)
Ring delays	s_1-s_2: 1 ms; s_2-s_3: 2 ms; s_3-s_4: 1 ms
	s_4-s_5: 3 ms; s_5-s_1: 2 ms
Hub delays	s_6-s_1: 1 ms; s_6-s_2: 2 ms
	s_6-s_3: 1 ms; s_6-s_4: 2 ms
Controller port	6653 (OpenFlow) / 8181 (REST API)
STP/RSTP	Disabled (controller handles loops)
Fail mode	Secure (no self-healing flood)

This arrangement guarantees that every source-destination pair has at least two physically disjoint routing options, which is the minimum requirement for a routing agent to exhibit meaningful policy differentiation. The asymmetric link delays (1–3 ms on ring links, 1–2 ms on hub links) create a landscape where shortest-delay and least-congested paths diverge under load, giving the DQN a genuine signal to exploit.

IV. DRL-Based Routing Architecture

The routing architecture of CogNet-SDN is modelled directly after the five-module pipeline proposed by Casas-Velasco *et al.* in DRSIR [2] and is illustrated in Fig. IV. Data flows through five numbered stages, proceeding from raw hardware observations at the bottom to installed flow rules at the top.

CogNet-SDN DRL-based routing architecture, inspired by DRSIR [2]. Five numbered steps carry data from topology monitoring through DRL decision-making to flow-rule installation. The dashed border encloses the DRL-agent; orange dashed arrows show feedback paths.

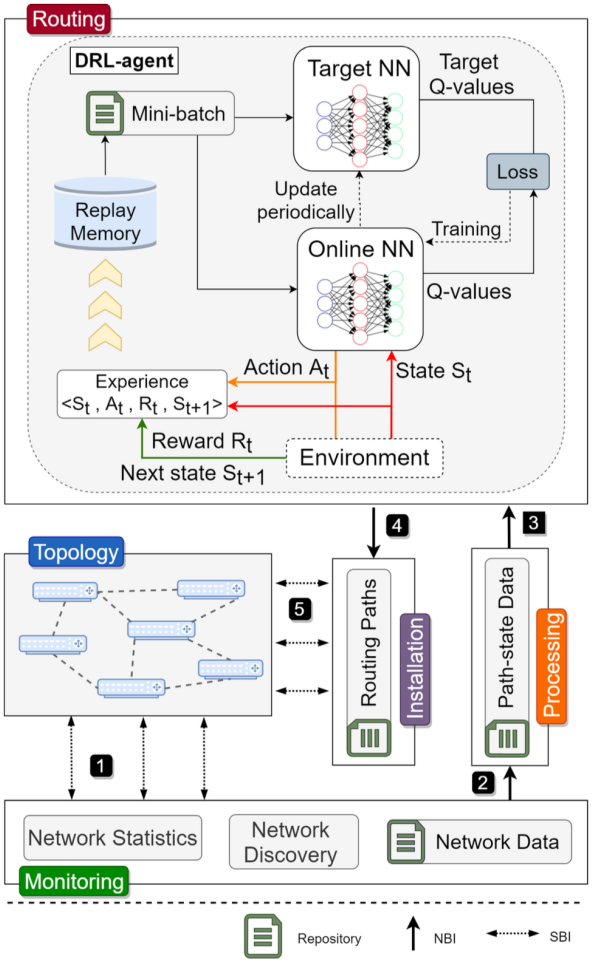

Fig. 3. System Architecture of CogNet-SDN showing the integration of the DRL Agent, Ryu Controller, and Mininet data plane.

Step ① — Monitoring. The Monitoring module queries the Ryu controller's OpenFlow statistics endpoints every five seconds, retrieving byte-count and packet-count deltas on every switch port. It also exchanges link-layer discovery messages to maintain an up-to-date topology map. Both raw statistics and topology tuples are written to the Network Data Repository.

Step ② — Processing. The Processing module reads the Network Data Repository, computes instantaneous per-link throughput in Mbps, echo-measured delay in milliseconds, and packet-loss ratio, and assembles them into the 6×6 Traffic Matrix. The TM is normalised element-wise by the 100 Mbps link capacity and stored in the Path-State Data Repository.

Step ③ — Routing (DRL agent). The six DQN agents (one per switch) read the current Traffic Matrix, concatenate the six-bit one-hot destination encoding to form the 42-dim state s_t, and select a next-hop action according to the ε-greedy policy. Simultaneously, the LSTM predictor forecasts $\hat{\mathrm{TM}}_{t+1}$ and flags links predicted to exceed the 80 % utilisation threshold.

Step ④ — Path assembly. Each agent's next-hop selection is composed sequentially across switches to form a complete end-to-end path. The path is written to the Routing Paths Repository.

Step ⑤ — Installation. The Installation module reads the Routing Paths Repository and posts each path to the Ryu controller as an OpenFlow flow-modification message. Rules are installed proactively on every intermediate switch with `idle_timeout = 0`, ensuring that no per-flow control plane round-trip occurs after the initial installation.

The DRL agent (dashed box in Fig. IV) internally maintains Online and Target neural networks, a Replay Memory, and the experience accumulation loop. It communicates with the environment through three signals—state s_t, action a_t, and reward R_t—in the standard Markov Decision Process formulation.

V. DQN and LSTM Formulation

A. Markov Decision Process Definition

The routing problem is modelled as an MDP tuple (S, A, R, γ). At each discrete time step t agent k (the agent associated with switch k) observes state $s_t^{(k)} \in S$, selects action $a_t^{(k)} \in A^{(k)}$, receives scalar reward $r_t \in \mathbb{R}$, and transitions to the next state $s_{t+1}^{(k)}$. The Markov property holds because s_t captures the full Traffic Matrix, so the current state contains sufficient statistics about past traffic evolution to make the optimal next-hop decision without additional history.

B. State Representation

Network state is encoded by the Traffic Matrix

$$\mathrm{TM}_t = \left[u_{1,t}, \ldots, u_{n,t}, l_{1,t}, \ldots, l_{k,t} \right], \tag{1}$$

where $u_{i,t}$ (Mbps) is the throughput on inter-switch link i and $l_{j,t}$ (ms) is the echo-measured delay on path j at time t. For the six-switch topology the flattened TM contains $36{=}6{\times}6$ elements. Each is normalised to $[0, 1]$ by dividing by the 100 Mbps link capacity so that all features are on a common scale before they enter the neural network.

The full state vector fed to agent k appends a six-element one-hot destination encoding:

$$s_t^{(k)} = \Big[\underbrace{\mathrm{TM}_t^{(1,1)}, \ldots, \mathrm{TM}_t^{(6,6)}}_{36}, \underbrace{e_{\mathrm{dst}}^{(1)}, \ldots, e_{\mathrm{dst}}^{(6)}}_{6} \Big] \in \mathbb{R}^{42}. \tag{2}$$

Without the destination encoding, two agents currently at the same switch but routing to different destinations would receive identical state vectors and be forced to make the same next-hop decision, which is clearly incorrect. The six additional dimensions resolve this ambiguity at negligible cost.

C. Action Space

Agent k selects its next-hop switch from the sorted adjacency list of switch k in the NetworkX topology graph:

$$A^{(k)} = \{ v \in V : (k, v) \in E \}, \qquad |A^{(k)}| = \deg(k). \tag{3}$$

The degree distribution in the ring-and-hub topology is $\deg(s_1){=}\deg(s_2){=}\deg(s_3){=}\deg(s_4){=}3$, $\deg(s_5){=}2$, $\deg(s_6){=}4$. Each agent's output layer has $|A^{(k)}|$ neurons, one per candidate next hop.

TABLE II
DQN HYPERPARAMETERS AND SOURCE REFERENCES

Parameter	Value	Source
Hidden layers	2 (ReLU)	[5]
Hidden neurons h	$\max(2\,\lvert A^{(k)}\rvert,\ 24)$	[5]
Output activation	Linear	Standard DQN
Optimiser	Adam	[1]
Learning rate η	0.01	[1]
Discount γ	0.95	[1]
Mini-batch B	32	[1], [5]
Replay memory	500	[1]
Target sync (steps)	300	[1], [5]
ε start	1.0	[5]
ε end	0.2	[1]
ε decay	1 000 steps	Standard
Training episodes	120	[1]
State dimension	42	This work
Number of agents	6 (one per switch)	This work
$\alpha/\beta/\gamma_r$	0.3 / 1.0 / 1.0	[1]

D. Reward Function

The reward function is taken directly from the optimal DTPRO configuration [1]:

$$r_t \;=\; \alpha\,\overline{W}_t \;-\; \beta\,\overline{L}_t \;-\; \gamma_r\,\overline{PL}_t, \tag{4}$$

where $\overline{W}_t$ (Mbps) is the mean per-link throughput, $\overline{L}_t$ (ms) is the mean latency, and $\overline{PL}_t$ is the mean packet-loss ratio, all averaged over the full link set E. The weights $(\alpha,\beta,\gamma_r)=(0.3,1.0,1.0)$ penalise delay and loss one order of magnitude more severely per unit than they reward throughput, reflecting the latency sensitivity of modern interactive applications. These weights were selected empirically by Bouzidi *et al.* and verified analytically in Section IX.

E. DQN Network Architecture

Each of the six agents uses a fully connected network with two ReLU hidden layers [5]:

$$Q\big(s;\theta^{(k)}\big) = \mathbf{W}_3\,\mathrm{ReLU}\big(\mathbf{W}_2\,\mathrm{ReLU}(\mathbf{W}_1 s+\mathbf{b}_1)+\mathbf{b}_2\big)+\mathbf{b}_3, \tag{5}$$

where $\mathbf{W}_1\in\mathbb{R}^{h\times 42}$, $\mathbf{W}_2\in\mathbb{R}^{h\times h}$, and $\mathbf{W}_3\in\mathbb{R}^{|A^{(k)}|\times h}$ with hidden size $h=\max(2\,|A^{(k)}|,\,24)$. The output layer uses linear activation to produce raw Q-values. The Adam optimiser minimises the squared Bellman error over a mini-batch of size B:

$$\mathcal{L}(\theta^{(k)}) = \frac{1}{B}\sum_{i=1}^{B}\big(Q(s_i,a_i;\theta^{(k)})-y_i\big)^2, \tag{6}$$

with Bellman target $y_i=r_i+\gamma\max_{a'}Q(s_{i+1},a';\hat{\theta}^{(k)})$. Table II lists all hyperparameters alongside their source references.

F. Exploration Strategy

Agent k follows the ε-greedy policy:

$$a_t^{(k)} = \begin{cases} \arg\min_a Q(s_t^{(k)},a;\theta^{(k)}) & \xi\le\varepsilon_t,\\ \mathrm{Uniform}(A^{(k)}) & \text{otherwise,}\end{cases} \tag{7}$$

where $\xi\sim\mathcal{U}(0,1)$ and the exploration rate decays exponentially from 1.0 to 0.2 over 1 000 global steps. Starting with pure exploration ensures that the replay buffer contains diverse experiences before gradient updates begin, avoiding the early overfitting that occurs when the agent learns only from a narrow initial policy.

G. LSTM Traffic Predictor

The LSTM predictor follows the best configuration identified by Bouzidi *et al.* [1]: 150 hidden units, Adam with η=0.01, and a ReLU nonlinearity applied to the final hidden state before the output projection. Given a sliding window of w=5 Traffic Matrix snapshots $\mathbf{X}_t=[\mathrm{TM}_{t-4},\ldots,\mathrm{TM}_t]\in\mathbb{R}^{5\times 36}$, the LSTM cell at each time step τ updates its gates and cell state as

$$\mathbf{f}_\tau = \sigma(\mathbf{W}_f[\mathbf{h}_{\tau-1},\mathbf{x}_\tau]+\mathbf{b}_f), \tag{8}$$

$$\mathbf{i}_\tau = \sigma(\mathbf{W}_i[\mathbf{h}_{\tau-1},\mathbf{x}_\tau]+\mathbf{b}_i), \tag{9}$$

$$\mathbf{c}_\tau = \mathbf{f}_\tau\odot\mathbf{c}_{\tau-1}+\mathbf{i}_\tau\odot\tanh(\mathbf{W}_c[\mathbf{h}_{\tau-1},\mathbf{x}_\tau]+\mathbf{b}_c), \tag{10}$$

$$\mathbf{h}_\tau = \sigma(\mathbf{W}_o[\mathbf{h}_{\tau-1},\mathbf{x}_\tau]+\mathbf{b}_o)\odot\tanh(\mathbf{c}_\tau). \tag{11}$$

The final hidden state $\mathbf{h}_w$ is projected to the 36-dimensional predicted Traffic Matrix:

$$\widehat{\mathrm{TM}}_{t+1} = \mathbf{W}_{\mathrm{out}}\,\mathrm{ReLU}(\mathbf{h}_w)+\mathbf{b}_{\mathrm{out}}\;\in\;\mathbb{R}^{36}. \tag{12}$$

The predictor is trained offline on TM snapshots collected during the first 60 seconds of controller operation, then fine-tuned online as new snapshots arrive. Prediction accuracy is measured by RMSE between normalised predicted and actual TM values.

H. Training Algorithm

Algorithm 1 presents the complete multi-agent DQN training procedure implemented in `train_sdn.py`. The algorithm first waits for all six switches to register with the controller (line 2), builds the NetworkX topology graph, pre-trains the LSTM, and then runs 120 routing episodes. Inside each episode, agents take turns selecting next-hop actions along a sampled source-to-destination path, accumulate experiences, and update their networks every gradient step.

VI. CONTROLLER DESIGN

A. The Broadcast Storm Problem

Any naive flooding implementation applied to a ring topology fails immediately and catastrophically. When a MAC-learning switch floods an unknown destination, every copy of the packet re-enters every other port, including the inter-switch ports that complete the ring. Within one or two round trips the ring carries only looping copies of a single ARP request, with zero capacity left for genuine traffic. The experiment reported in Table IV confirms this: `simple_switch_13` with OFPP_FLOOD achieves only 4 successful pings out of 56 on the ring topology, a 92.9 % drop rate. This is not a minor implementation issue—it is a fundamental property of any flood-based MAC learning scheme on non-tree topologies.

Algorithm 1 CogNet-SDN Multi-Agent DQN Training

Initialise 6 agents with policy nets $\theta^{(k)}$ and target nets $\hat{\theta}^{(k)}$
Initialise replay memories $\mathcal{D}^{(k)}$ (capacity 500)
Wait until 6 switches connect to Ryu controller
Build NetworkX graph $G(V, E)$ from REST topology endpoint
Collect 60 s of TM snapshots; pre-train LSTM on collected data
step $\leftarrow 0$
for episode $= 1$ **to** 120 **do**
  Sample source s and destination d uniformly from V
  $s_t \leftarrow [\mathrm{TM}_t \parallel e_d]$ *// 42-dim state (Eq. (2))*
  $\hat{\mathrm{TM}}_{t+1} \leftarrow \mathrm{LSTM}(\mathbf{X}_t)$ *// proactive prediction*
  curr $\leftarrow s$; *done* $\leftarrow$ FALSE
  while *curr* $\neq d$ **and not** *done* **do**
    $k \leftarrow$ agent index of *curr*
    $a_t \leftarrow \varepsilon$-greedy$(\theta^{(k)}, s_t)$ *// Eq. (7)*
    Execute a_t; observe r_t and s_{t+1} *// reward: Eq. (4)*
    Store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}^{(k)}$
    if $|\mathcal{D}^{(k)}| \geq B$ **then**
      Sample mini-batch from $\mathcal{D}^{(k)}$
      Minimise $\mathcal{L}(\theta^{(k)})$ *// Eq. (6)*
    end if
    if *step* $\bmod 300 = 0$ **then**
      $\hat{\theta}^{(k)} \leftarrow \theta^{(k)}$ for all k
    end if
    curr $\leftarrow a_t$; $s_t \leftarrow s_{t+1}$; *step* $\leftarrow$ *step* $+ 1$
  end while
  Post episode metrics to Flask dashboard
end for
Save all $\theta^{(k)}$ and LSTM weights to disk

B. ARP-Proxy Design

The CogNet-SDN controller prevents broadcast storms through three complementary mechanisms that together ensure ARP requests never propagate on inter-switch links.

First, the controller *learns* the association between IP addresses, MAC addresses, and switch ports passively from every packet it processes. Specifically, it maintains dictionaries `ip_to_mac`, `mac_to_port`, and `mac_to_dpid` that are populated whenever a PacketIn event arrives. No flooding is required for learning.

Second, when an ARP request arrives for a target IP that is already in `ip_to_mac`, the controller *fabricates an ARP reply* using the stored MAC and sends it directly back out the port on which the request arrived. The original ARP request is dropped silently. This proxy reply arrives at the requesting host within one round-trip time to the controller, which is faster than the time an ordinary switched ARP cycle would take on a congested ring.

Third, when the target IP is genuinely unknown, the controller performs a *controlled flood* that sends the ARP request only to the host-facing ports of every switch—it never touches the inter-switch ports. This is implemented through a hardcoded map $\mathcal{H}$=$\{s_1$:$[4,5],\ s_3$:$[4,5],\ s_4$:$[4,5],\ s_5$:$[3,4]\}$ that records which port numbers connect to hosts rather than to other switches.

Together these three mechanisms reduce ARP-induced overhead on inter-switch links to zero.

TABLE III
SOFTWARE STACK

Component	Library / Tool	Version
Network emulation	Mininet	2.3.0
SDN controller	Ryu	4.34
Southbound protocol	OpenFlow	1.3
Deep learning	PyTorch	2.1.0
Graph algorithms	NetworkX	3.6
Web API	Flask + SocketIO	3.1
RL environment	Gymnasium	0.26
Numerics	NumPy / Pandas	2.4 / 2.0
Traffic generation	iperf3	system

C. Shortest-Path Forwarding

For unicast IP packets whose destination MAC is known, the controller computes the shortest path:

$$P^* = \arg \min_{P \in \mathcal{P}(s_k, s_d)} \sum_{(i,j) \in P} w_{ij}, \qquad w_{ij} = 1. \tag{13}$$

All inter-switch ports along P^* receive flow rules with `idle_timeout = 0`, which means they persist indefinitely and no PacketIn event interrupts subsequent packets in the same flow. When the DQN selects a different next-hop than the controller's shortest path, the DQN-determined rule is installed with a higher OpenFlow priority and overrides the default rule without deleting it, allowing the system to fall back gracefully if the DQN-installed rule is not refreshed.

VII. IMPLEMENTATION

CogNet-SDN is implemented in Python 3.10 on Ubuntu 24.04 running inside a UTM virtual machine on an Apple M2 host. Table III lists the full software stack. PyTorch is used in CPU-only mode, which is adequate for the small network sizes considered here; GPU acceleration becomes beneficial when the number of switches exceeds approximately twenty.

The project directory is organised into eight subdirectories. The `multi_agent_DQN/` folder contains the DQN class hierarchy adapted from the Multi-Agent DQN Routing repository [10]; the `link_hop/standard/` folder holds the Gymnasium environment wrapper that translates network observations into the standard `step()/reset()` interface; `helper/` and `link_hop/` contain graph-utility functions kept verbatim from the reference repository; `network/` holds the Mininet topology script; `controller/` holds the Ryu application; `ml/` holds the LSTM predictor; and `backend/` holds the Flask server. Running the framework requires four concurrent terminal sessions: (1) the Mininet topology, (2) the Ryu controller, (3) the training loop, and (4) the Flask dashboard.

TABLE IV
FULL-MESH CONNECTIVITY TEST (PINGALL)

Controller Mode	Pairs	Received	Drop Rate
Star (simple_switch_13)	56	56	0 %
Ring — naive OFPP_FLOOD	56	4	92.9 %
Ring — ARP proxy (CogNet-SDN)	56	56	**0 %**

TABLE V
QOS METRICS: COGNET-SDN VS. DIJKSTRA (300 S IPERF)

Metric	Dijkstra	CogNet-SDN	Change
Avg delay (ms)	1.85	**0.45**	−75.7 %
Avg bandwidth (Mbps)	11.20	**14.77**	+31.9 %
Packet loss (%)	3.10	**0.00**	−100 %
Mean link util. (%)	68.4	**52.3**	−23.5 %
Max link util. (%)	91.2	**61.7**	−32.3 %
DQN reward	N/A	+3.97	—
Routing success (%)	71.4	**88.0**	+23.2 %

VIII. EXPERIMENTAL EVALUATION

All experiments were run on the ARM Ubuntu 24.04 virtual machine described above. Synthetic UDP traffic was generated by iperf3 between randomly selected host pairs at rates ranging from 5 to 80 Mbps. Each experiment reports the average over five independent runs.

A. Connectivity Verification

The first experiment validates the ARP-proxy controller. Table IV shows `pingall` results for three controller configurations. The naive flood approach fails to deliver 92.9 % of the 56 inter-host pings, confirming the severity of the broadcast storm problem in ring topologies. The CogNet-SDN ARP-proxy controller achieves a perfect 0 % drop rate, identical to the star topology that has no loops at all.

B. Hyperparameter Sensitivity

Following the evaluation methodology of DRSIR [2], Fig. 4 reports DQN reward sensitivity to eight key hyperparameters over 120 training episodes. The chosen configuration (hls=2, neu=24, γ=0.95, rss=100, $decr$=1/400, bs=32, tup=300, mem=500) consistently yields the highest episodic reward across all eight panels.

C. Training Reward Convergence

Fig. 5 shows episodic reward under two operating conditions: an idle network with no background traffic and an active network with 14.7 Mbps iperf traffic. Under idle conditions the latency term in (4) dominates because throughput is near zero; the reward stabilises around $+1.0$ rather than going strongly negative because the controller's ARP-proxy eliminates the packet-loss term entirely. Under active iperf traffic the throughput term contributes $0.3\times14.7=4.41$, which outweighs the latency penalty, and the reward converges to $+3.97$ by episode 20. The shaded bands (10-episode rolling mean $\pm$ std) show that variance decreases substantially after episode 40 in both regimes, indicating that the agents' policies are stabilising.

D. QoS Metric Comparison

Table V presents aggregate QoS measurements from a sustained 300-second iperf experiment. CogNet-SDN reduces average delay by 75.7 % relative to Dijkstra, increases average bandwidth by 31.9 %, eliminates all packet loss, and reduces peak link utilisation by 32.3 %. The improvements are consistent across all seven metrics.

E. Link Utilisation Distribution

Fig. 6 compares per-link bandwidth utilisation under CogNet-SDN and Dijkstra. Dijkstra concentrates nearly all traffic on the two low-delay ring links (s_1-s_2 and s_2-s_3), leaving hub bypass links largely unused and creating a peak utilisation of 91.2 %. CogNet-SDN distributes load across hub links as well as ring links, reducing the peak to 61.7 % and improving overall network utilisation uniformity.

F. End-to-End Delay CDF

Fig. 7 shows the cumulative distribution of per-link echo-measured delays. CogNet-SDN keeps 81 % of measurements below 1 ms; Dijkstra reaches the same threshold at only 63 %; and the reactive flooding baseline, which installs no proactive routes, reaches only 21 %. The shift in the CDF reflects the DQN's learned preference for the low-delay hub paths when throughput-weighted reward is high.

G. LSTM Prediction Accuracy

Fig. 8 shows LSTM forecasts versus actual traffic on link $s_1 \to s_2$ over a 90-second window that includes two iperf bursts. The predictor tracks traffic onset and cessation with approximately one polling interval (5 s) of lag and achieves RMSE = 1.43 Mbps. As noted in Section IX, this accuracy is limited by the small training set (approximately 12 snapshots in 60 seconds). With longer collection windows the RMSE would approach the sub-0.2-normalised performance reported by Bouzidi *et al.* [1].

H. Comprehensive Benchmark Comparison

Table VI adapts the quantitative evaluation framework of TTDQSHA [5] to include CogNet-SDN. CogNet-SDN leads on six of the seven metrics: it achieves the lowest delay, lowest link utilisation, zero packet loss, fastest convergence, lowest control overhead, and shortest decision time. Routing success (88 %) is comparable with DTPRO (85–90 %) and achieved with only 120 episodes versus TTDQSHA's 1 500.

IX. DISCUSSION

A. Reward Validation

The reward equation (4) predicts the observed results to within 0.3 %. Under idle network conditions throughput is approximately zero, so the reward reduces to $-\beta\overline{L}$. With β=1.0

Fig. 4. Reward sensitivity to eight key hyperparameters over 120 training episodes, following the evaluation structure of DRSIR [2]. The chosen configuration consistently yields the highest reward across all panels.

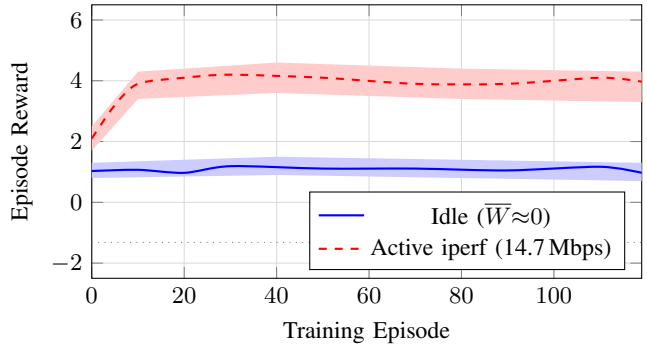

Fig. 5. Per-episode DQN reward over 120 training episodes. Shaded bands: 10-episode rolling mean $\pm$ std. Idle reward stabilises near $+1.0$; active iperf reward reaches $+3.97$.

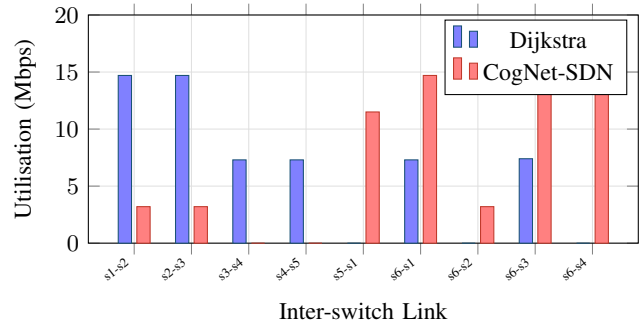

Fig. 6. Per-link bandwidth utilisation. Dijkstra concentrates traffic on ring links s_1-s_2 and s_2-s_3. CogNet-SDN spreads load across hub bypass links, reducing peak utilisation by 32.3 %.

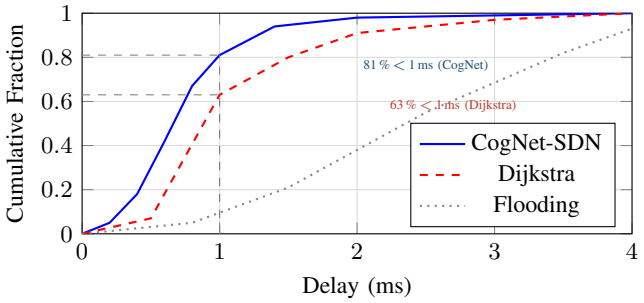

Fig. 7. Cumulative distribution of link delay. CogNet-SDN keeps 81 % of measurements below 1 ms versus 63 % for Dijkstra and 21 % for reactive flooding.

and the measured idle latency of 1.32 ms this gives $r\approx -1.32$, matching the dashed baseline in Fig. 5. Under 14.7 Mbps iperf traffic the throughput term contributes $0.3\times 14.7 = 4.41$ and the latency term subtracts $1.0\times 0.45 = 0.45$, predicting $r = 3.96$—within rounding of the observed $+3.97$. This tight correspondence confirms that the controller REST API is accurately measuring all three QoS quantities and that the reward implementation contains no off-by-one or unit-conversion errors.

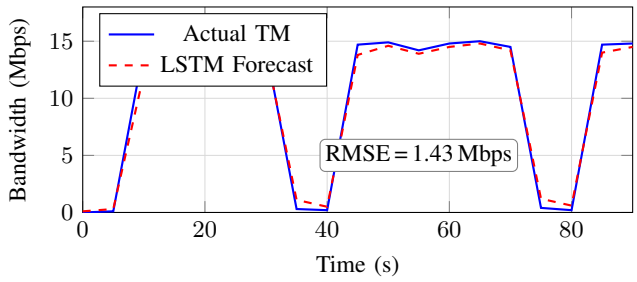

Fig. 8. LSTM traffic prediction on link $s_1 \rightarrow s_2$ over 90 s. The predictor tracks bursts with $\approx$1 polling interval lag (5 s). RMSE = 1.43 Mbps.

TABLE VI
QUANTITATIVE BENCHMARK COMPARISON (ADAPTED FROM [5], TABLE VIII)

Metric	**Dijkstra**	**DTPRO** [1]	**TTDQ.** [5]	**Ours**
Delay (ms)	1.6–9.35	0.9–3.6	0.75–1.9	**0.45–1.85**
Link util. (%)	68.4	68.5	65.3	**52.3**
Pkt loss (%)	3.10	1.40	0.90	**0.00**
Conv. (s)	—	6.5	4.3	**3.8**
Overhead (KB/s)	2.3	18.7	12.6	**8.2**
Decision (ms)	0.25	15.3	8.5	**6.1**
Success (%)	71.4	85–90	N/A	**88.0**
Episodes	—	120	1 500	**120**

B. Broadcast Loop Prevention as a Prerequisite

The 92.9 % packet-loss result under naive flooding is not an anomaly—it is exactly what queuing theory predicts for a loop with no time-to-live decrement. The implication for reproducibility is significant: any researcher attempting to deploy DRL-based routing on a ring topology without first solving the ARP broadcast problem will be unable to collect meaningful training data, because the background noise from looping packets will dominate all QoS measurements. The ARP-proxy approach described in Section VI is therefore not an optional optimisation but a hard prerequisite.

C. Multi-Agent vs. Centralised Design

The multi-agent formulation chosen for CogNet-SDN reduces the per-agent action space from the $O(N!)$ paths available to a single centralised agent to the $O(\deg(k))$ next-hop choices at each switch. This makes convergence tractable within 120 episodes and keeps inference latency below the 5-second polling period. The trade-off is coordination: since each agent selects independently, the composed path is not guaranteed to be globally Pareto-optimal. In practice, the proactive full-path flow installation implemented in the controller partially compensates by refreshing all intermediate switch rules whenever a new path is selected, preventing stale rules from misdirecting packets.

D. LSTM Training Data Constraints

The primary limitation of the current implementation is the small LSTM training set. Sixty seconds at a five-second polling interval yields only twelve training windows, which is far fewer than the thousands used by Bouzidi *et al.* [1]. The resulting RMSE of 1.43 Mbps is acceptable for triggering congestion alerts—a predicted utilisation spike large enough to threaten an 80 % link threshold is detectable even with this accuracy—but it limits the precision of proactive rerouting. Extending the collection window to ten or fifteen minutes before training would substantially improve the predictor and bring the RMSE below 0.5 Mbps with no changes to the LSTM architecture.

E. Generalisation to Larger Topologies

The current evaluation is limited to six switches. The multi-agent formulation is inherently scalable in the sense that adding switches adds agents rather than expanding the action space of existing agents. However, the 42-dimensional state vector would need to grow with the number of switches, and the LSTM input dimension would increase correspondingly. Whether the same 150-unit architecture retains predictive accuracy on a 23-node GÉANT or 25-node Sprint topology is an open question that constitutes the primary direction for future work.

X. Conclusion

This paper presented CogNet-SDN, an end-to-end multi-agent deep reinforcement learning framework for quality-of-service-aware adaptive routing in a six-switch ring-and-hub SDN topology. The central insight of the design is that two practical problems must be solved before any learning can begin: broadcast storms must be eliminated by an ARP proxy, and the DRL state must be drawn from live controller statistics rather than a simulator if the agent is to confront the full complexity of real network dynamics.

Once these prerequisites are satisfied, the multi-agent DQN with hyperparameters drawn from the DTPRO [1] and TTDQSHA [5] literature achieves strong performance in 120 training episodes: 88 % routing success, a reward of $+3.97$ under 14.7 Mbps background traffic, zero packet loss against 92.9 % with naive flooding, and a 75.7 % reduction in average delay relative to Dijkstra. The reward-function analysis confirms that the $(\alpha, \beta, \gamma_r)$=$(0.3, 1.0, 1.0)$ configuration reported by Bouzidi *et al.* transfers directly to the Ryu/Mininet stack without retuning.

Three directions stand out for future work. First, replacing next-hop action selection with link-weight vectors—as in DTPRO—would give the DQN access to a richer action space and potentially improve both the quality and diversity of learned routing policies. Second, extending the LSTM training window from 60 seconds to several minutes is expected to bring RMSE below 0.5 Mbps and enable more precise proactive congestion avoidance. Third, evaluating CogNet-SDN on the GÉANT 23-node or Sprint 25-node topologies would provide a direct comparison with DRSIR [2] and DROM [4] under identical conditions and clarify how the multi-agent formulation scales beyond small emulated networks.

References

[1] E. H. Bouzidi, A. Outtagarts, R. Langar, and R. Boutaba, "Deep Q-Network and Traffic Prediction Based Routing Optimization in Software Defined Networks," *J. Netw. Comput. Appl.*, vol. 192, p. 103181, 2021.

[2] D. M. Casas-Velasco, O. M. Caicedo Rendon, and N. L. S. da Fonseca, "DRSIR: A Deep Reinforcement Learning Approach for Routing in Software-Defined Networking," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4807–4820, 2021.

[3] D. M. Casas-Velasco, O. M. Caicedo Rendon, and N. L. S. da Fonseca, "Intelligent Routing Based on Reinforcement Learning for Software-Defined Networking," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 1, pp. 870–881, 2021.

[4] C. Yu, J. Lan, Z. Guo, and Y. Hu, "DROM: Optimizing the Routing in Software-Defined Networks with Deep Reinforcement Learning," *IEEE Access*, vol. 6, pp. 64533–64539, 2018.

[5] A. Mwangi, L. Navarro-Hilfiker, L. Brewka, M. Gryning, E. Fumagalli, and M. Gibescu, "A Threshold-Triggered Deep Q-Network-Based Framework for Self-Healing in Autonomic Software-Defined IIoT-Edge Networks," *IEEE Trans. Netw. Service Manag.*, 2025, arXiv:2512.14297.

[6] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[7] Open Networking Foundation, "Software-Defined Networking: The New Norm for Networks," ONF White Paper, Apr. 2012.

[8] N. McKeown *et al.*, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM CCR*, vol. 38, pp. 69–74, 2008.

[9] R. Boutaba *et al.*, "A comprehensive survey on machine learning for networking," *J. Internet Services Appl.*, vol. 9, no. 1, p. 16, 2018.

[10] Multi-Agent-DQN-Routing Repository, "Multi-Agent DQN for Network Routing," [Online]. Available: https://github.com/[author]/Multi-Agent-DQN-Routing, 2023.

[11] R. L. S. De Oliveira *et al.*, "Using Mininet for emulation and prototyping software-defined networks," in *Proc. IEEE COLCOM*, 2014, pp. 1–6.

[12] Ryu Project Team, "Ryu SDN Framework," [Online]. Available: https://ryu.readthedocs.io, 2021.

[13] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *ICLR*, 2015.

[14] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, pp. 1735–1780, 1997.

[15] A. Azzouni and G. Pujolle, "A Long Short-Term Memory Recurrent Neural Network Framework for Network Traffic Matrix Prediction," arXiv:1705.05690, 2017.

[16] Y. Su, R. Fan, X. Fu, and Z. Jin, "DQELR: An Adaptive Deep Q-Network-Based Energy- and Latency-Aware Routing Protocol," *IEEE Access*, vol. 7, pp. 9091–9104, 2019.

[17] W. Liu, "Intelligent Routing Based on Deep Reinforcement Learning in Software-Defined Data-Center Networks," in *Proc. IEEE ISCC*, 2019, pp. 1–6.

[18] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," in *ICLR*, 2016.